

Math Handwriting Recognition: Data Processing, Model Development, and Evaluation

Ha Manh Tuan¹, Tran Xuan Bao Viet², Nguyen Vo Minh Dat³,
Vo Minh Nhat⁴

Artificial Intelligence, FPT University Ho Chi Minh, Ho Chi Minh,
Vietnam.

Contributing authors: tuanhmse180249@fpt.edu.vn;
viettxbqe180024@fpt.edu.vn; datnvmqe180070@fpt.edu.vn;
nhatvmse173616@fpt.edu.vn;

Abstract

The automated conversion of handwritten mathematical expressions into computable formats is a critical component of document digitization and intelligent tutoring systems. This paper details the data processing pipeline, model development, and empirical evaluation of a mathematical handwriting recognition system. Utilizing the CROHME 2019 dataset combined with a self-curated synthetic corpus, we conducted structural graph transformations to preserve spatial syntax. Our experimental framework evaluates a baseline Vision Transformer (ViT) Hybrid against a multimodal Qwen3-VL model adapted via Low-Rank Adaptation (LoRA). Empirical results demonstrate that the fine-tuned Qwen3-VL model significantly outperforms the baseline, achieving an exact match accuracy of 43.74% and a BLEU-4 score of 77.67, confirming its robustness in parsing complex spatial relationships in mathematical formulations.

Keywords: Handwriting Recognition, Mathematical Expressions, Vision-Language Models, Qwen3-VL, CROHME

1 Introduction

The reliable translation of handwritten mathematical expressions into standard computable formats (e.g., LaTeX) remains a complex challenge in computer vision and pattern recognition [1]. This study proposes a machine learning framework capable

of recognizing and parsing handwritten mathematical expressions, ranging from basic arithmetic to complex hierarchical structures including exponents and nested roots. By leveraging advanced vision-language models, the system directly maps handwritten trajectories and visual features into structured mathematical syntax.

2 Dataset and Preprocessing

2.1 Data Source

The primary dataset utilized in this study is derived from the ICDAR 2019 Competition on Recognition of Handwritten Mathematical Expressions (CROHME) [2]. The data contains temporally classified labels generated from Symbol Label Graphs. The dataset comprises two main components:

- **InkML files:** XML-based documents containing stroke-level handwriting data captured from digital devices [3].
- **Annotation files:** Ground-truth labels corresponding to the structural elements of each handwriting sample.

There are two parts:

- Inkml files: XML based files containing handwriting data (sequences of handwritten strokes) captured from the devices.

```
dataset
├── crohme2019
│   └── crohme2019
│       ├── test
│       ├── train
│       └── valid
```

- Annotation files: Containing the labels for each input handwriting samples.

```
dataset
├── crohme2019_test.txt
├── crohme2019_train.txt
└── crohme2019_valid.txt
```

Fig. 1 Directory structure of the InkML and Annotation files within the dataset.

2.2 Synthetic Data Generation and Annotation

To augment the primary training corpus and enhance the model’s robustness against diverse handwriting styles, a supplementary dataset comprising 2,000 synthetic samples was procedurally generated and manually annotated by the research team.

The data augmentation pipeline initiates with a custom automated script (`gen_latex.py`) designed to synthesize a diverse array of mathematical LaTeX formulas. As illustrated in Figure 3, these formulas are rendered into static reference images, systematically collected, and partitioned into sub-batches to streamline the subsequent annotation workflow.

To bridge the modality gap between static rendered images and online handwriting trajectories, a proprietary Graphical User Interface (GUI) annotation tool was

An example of inkml file, the points (x, y coords) are records in each trace (handwritten stroke).

```
<ink xmlns="http://www.w3.org/2003/InkML">
  <traceFormat>
    <channel name="X" type="decimal"/>
    <channel name="Y" type="decimal"/>
  </traceFormat>
  ...

  <trace id="0">
    241 123, 240 123, 239 123, 238 123, 237 123, 236 123, 236 122, 237 122, 238 122, 240 122, 24
    2 122, 244 122, 247 122, 251 121, 254 121, 257 121, 260 120, 262 120, 265 120, 266 120, 267
    120, 268 120, 269 120, 268 120
  </trace>
```

Fig. 2 Sample expression paired with its corresponding ground-truth label graph.

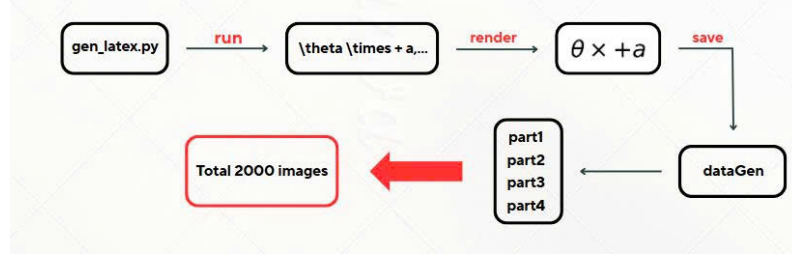


Fig. 3 Pipeline for synthetic LaTeX formula generation and static image rendering.

developed. The application displays the synthesized mathematical formula as a visual reference on the left panel, while providing an interactive digital canvas on the right (Figure 4).

Annotators manually transcribe the reference expression onto the canvas using digital drawing inputs. During this process, the system dynamically captures the sequential spatio-temporal coordinates of the pen strokes. Upon completion, the application automatically structures and exports the trajectory data into the standardized XML-based InkML format [3], accurately generating the corresponding `<traceGroup>` and `<trace>` tags. This rigorous self-curation process guarantees a controlled, high-quality expansion of the dataset, tailored specifically to mathematical syntax parsing.

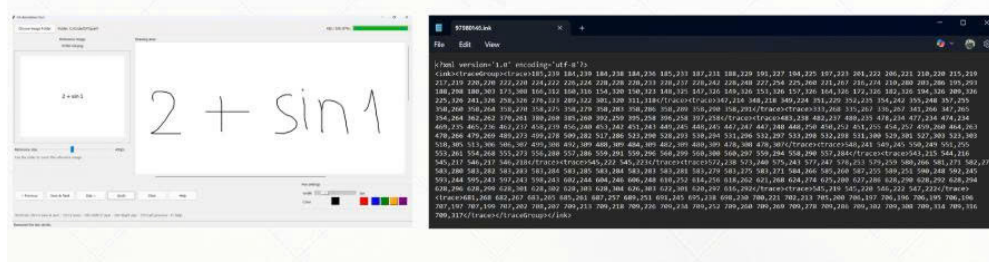
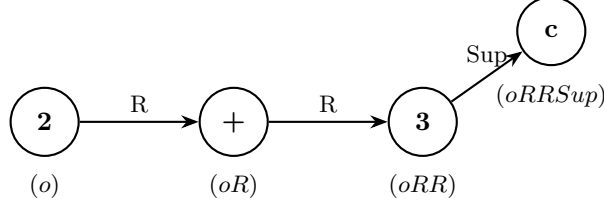


Fig. 4 Custom annotation tool interface utilized for capturing manual stroke trajectories and exporting directly to the standard InkML format.

2.3 Data Preprocessing

During preprocessing, mathematical expressions are transformed into a **Symbol Label Graph (SLG)** representation. The SLG is a directed graph structure designed to preserve both spatial and structural dependencies among mathematical symbols:

- **Nodes** represent distinct mathematical symbols (e.g., x , $+$, 2 , $=$).
- **Edges** represent spatial or structural relationships between symbols, categorized as “right”, “left”, “above”, “below”, or “superscript”.



This SLG encoding retains the two-dimensional layout of formulations, enabling downstream models to parse syntactic structures rather than relying on sequential linear text estimation.

INKML Format: The Ink Markup Language (InkML) format records the trajectory of pen strokes as continuous sequences of (x, y) coordinates over time [3]. Multiple traces forming a semantic unit (e.g., strokes constituting a single symbol) are logically grouped using the `traceGroup` element.

```
<trace id="0">39,103 39,102 38,102 38,101 37,101 36,101</trace>
```

Fig. 5 Extract of an InkML file detailing stroke trace coordinates.

```
<traceGroup xml:id="grp1">
  <annotation type="truth">x</annotation>
  <traceView traceDataRef="0"/>
  <traceView traceDataRef="1"/>
</traceGroup>
```

Fig. 6 Implementation of the `traceGroup` element for stroke association.

3 Exploratory Data Analysis

3.1 Data Overview

Table 1 outlines the distribution of tokens across the predefined data splits. The base training set constitutes approximately 82% of the corpus.

Table 1 Corpus distribution and token statistics across dataset splits.

Split	Expressions	Avg. Tokens	Median Tokens	Max Tokens	Min Tokens
Test	1199	19.91	17	91	0
Train	8900	18.57	15	133	1
Valid	986	19.32	17	189	1

Statistical analysis reveals that the average token length centers around 18–20, denoting moderate formulation complexity. The maximum token length reaches 189, highlighting the presence of highly intricate, multi-line expressions which inherently challenge standard sequence modeling architectures.

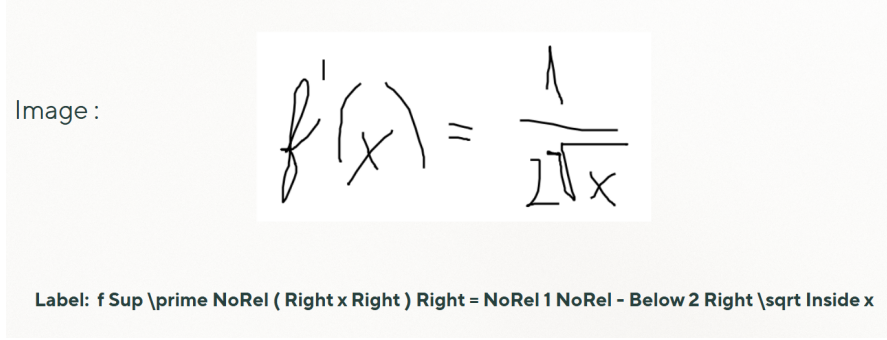


Fig. 7 Visual representation of data sample complexity.

3.2 LaTeX Token Frequency

Frequency analysis demonstrates that specific spatial tokens, notably **Right** (58,090 occurrences) and **NoRel** (22,130 occurrences), dominate the dataset. This pronounced imbalance underscores the critical role of structural relationship modeling over standalone symbol classification [1].

3.3 Stroke Statistics and Correlation

The mean stroke count per expression is approximately 14. Similar distributions across training, validation, and testing partitions confirm consistent sampling regarding expression complexity (Table 2 and Figure 9). Furthermore, scatter plot analysis

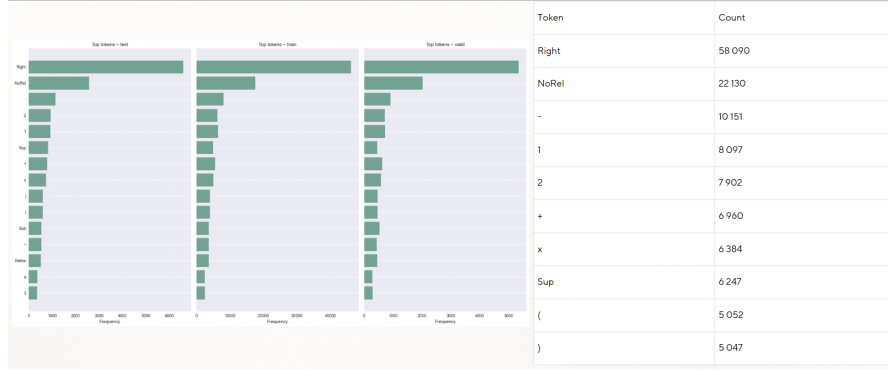


Fig. 8 Frequency distribution of LaTeX structural and arithmetic tokens.

(Figure 10) illustrates a strong positive linear correlation between LaTeX token counts and physical stroke volume.

Table 2 Stroke volume statistics per dataset split.

Split	Samples	Avg. Traces	Median Traces	Max Traces
Test	1199	14.41	12	68
Train	8900	13.84	12	99
Valid	986	13.99	12	115

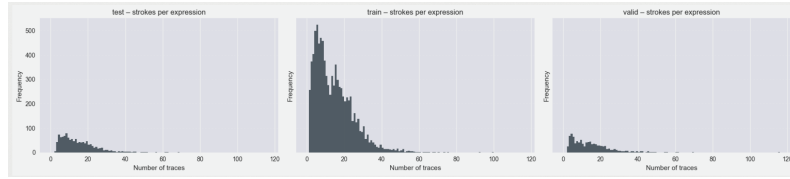


Fig. 9 Histogram depicting trace (stroke) frequency distribution.

4 Build Model

4.1 Introduction

To evaluate the effectiveness of different modeling approaches for handwritten mathematical expression recognition, two main strategies were implemented:

- **Method 1** - Vision Transformer Hybrid (Own Built Model) [5].
- **Method 2** - Fine-Tuned Qwen3-VL (Unsloth Framework) [7].

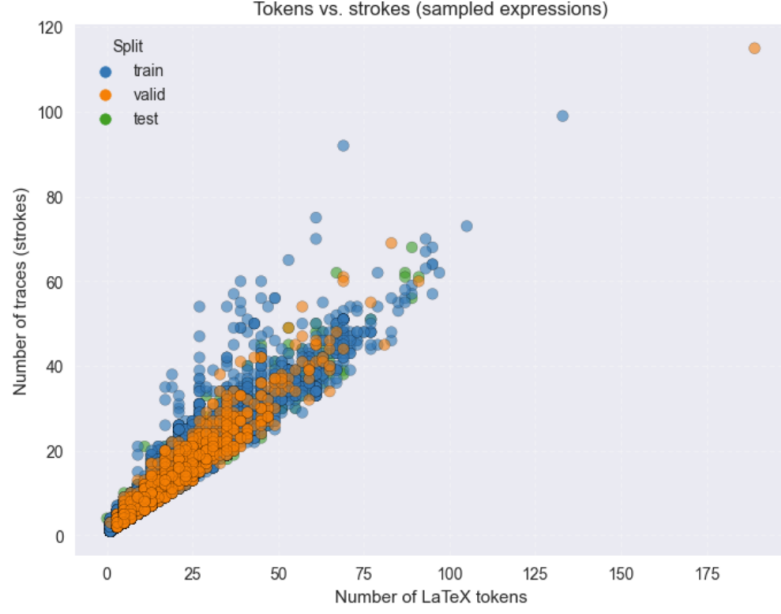


Fig. 10 Correlation between LaTeX tokens and physical writing strokes.

4.2 Vision Transformer Hybrid

4.2.1 Architecture Overview

The **Vision Transformer Hybrid (ViT Hybrid)** model combines convolutional and transformer-based representations to leverage both local and global visual dependencies [5, 6].

- **Encoder:**

- **Backbone:** BiT ResNetv2 + ViT (384×384) [6].
- **Hybrid base:** google/vit-hybrid-base-bit-384
- **Hidden size:** 768
- **Number of layers:** 12

- **Decoder:**

- **Type:** TransformerDecoder
- $d_{\text{model}} = 768$, $n_{\text{head}} = 8$
- **Number of decoder layers:** 4
- **Feedforward dimension:** 1024

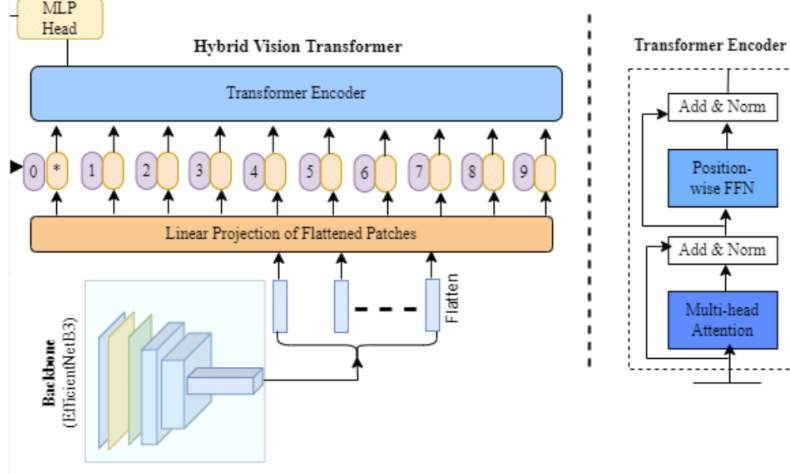


Fig. 11 Vision Transformer Hybrid model Structure.

4.2.2 Training Process

The model was trained for **10 epochs** with a **batch size of 8** and a **learning rate of 5e-5**. The training used the `CrossEntropyLoss` function and the `AdamW` optimizer for stable convergence.

During training, both the training and validation losses decreased steadily, indicating effective learning. The validation loss stabilized after several epochs, suggesting good generalization and minimal overfitting.

- **Epochs:** 10
- **Batch size:** 8
- **Learning rate:** 5e-5
- **Loss function:** `CrossEntropyLoss`
- **Optimizer:** `AdamW`

4.3 Fine-tune Qwen3VL

The objective of this stage is to adapt the Qwen3-VL model to recognize handwritten mathematical expressions from the CROHME dataset [2]. The model is fine-tuned using the Unsloth framework [8], which integrates LoRA [9], QLoRA [10], and 4-bit quantization to reduce computational cost while maintaining performance.

4.3.1 Evaluate on CROHME2019 test dataset before fine-tuning

In this stage, the pre-trained **Qwen3VL** model was evaluated on the **CROHME2019 test dataset** to analyze its initial performance before any fine-tuning.

Qwen3VL (Pre-trained model):

As shown in the figure above, the base model can recognize the overall structure of the handwritten mathematical expression. However, it still produces small structural

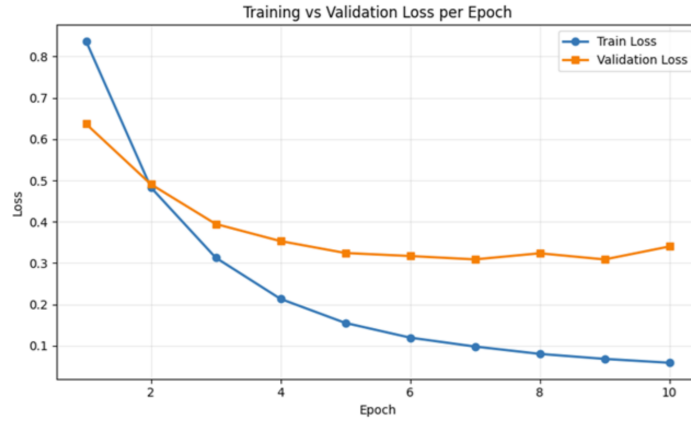


Fig. 12 Training and validation loss curves of the Vision Transformer Hybrid model.

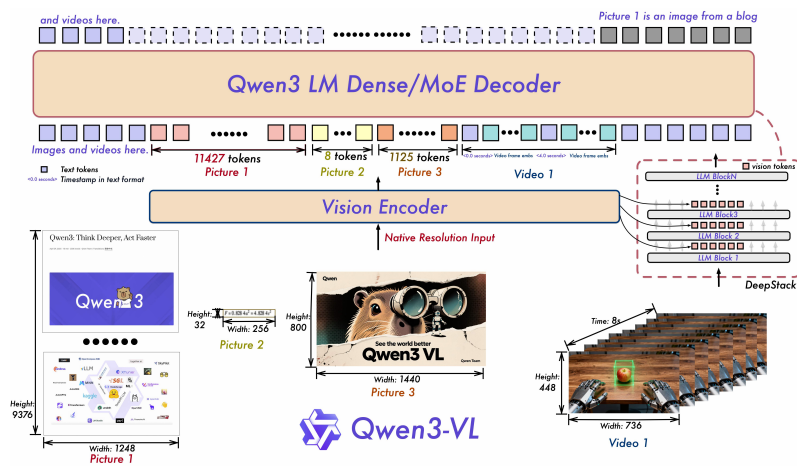


Fig. 13 Qwen3VL - Model Architecture.

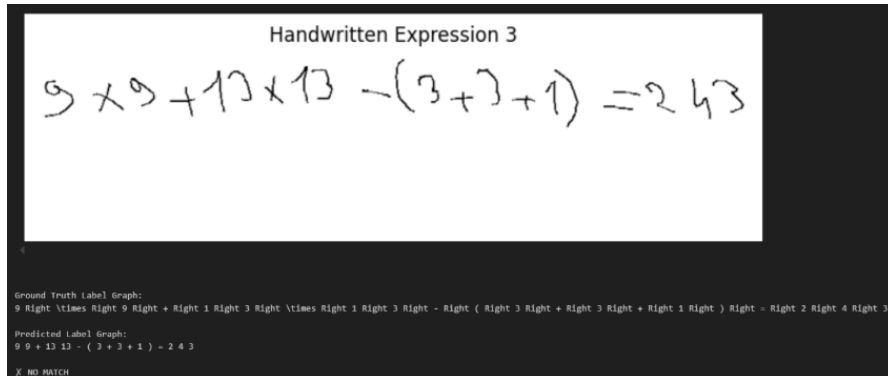


Fig. 14 Output of the pre-trained Qwen3VL model showing structural mismatches.

and relational mismatches in the *Predicted Label Graph* compared to the *Ground Truth Label Graph*, leading to a classification outcome of: **X NO MATCH**.

After that, the model underwent fine-tuning on domain-specific handwritten mathematical data to enhance its understanding of symbol relationships and spatial dependencies.

4.3.2 Framework

- Unsloth (LoRA + QLoRA + 4-bit quantization) was applied to reduce memory usage while preserving model accuracy [8, 10].
- The setup trains only about 1% of total parameters using LoRA adapters (rank = 16), enabling efficient adaptation to the mathematical handwriting recognition task [9].

4.3.3 Setup

- **Base model:** unsloth/Qwen3-VL-4B-Instruct-bnb4bit
- **Enabled modules:** Vision + Language + Attention + MLP layers
- **LoRA configuration:** $r = 16$ (lightweight fine-tuning)
- **Training environment:** Low-memory setup with 4-bit quantization

4.3.4 Data

- **Dataset:**
 - 8.9K InkML files from CROHME2019 dataset [2].
 - Additional 2K augmented samples generated using data augmentation.
- **Preprocessing:** Each record was converted into a conversational input format:
 - **User:** [image + instruction]
 - **Assistant:** [label graph]
- This structure allows Qwen3-VL to map visual mathematical expressions to corresponding LaTeX-style label graph.

4.3.5 Training Configuration

- **Trainer:** SFTTrainer combined with UnslothVisionDataCollator
- **Batch size:** 8
- **Learning rate:** $2e-4$
- **Epochs:** 1
- **Warmup steps:** 5
- **Evaluation:** Every 20 steps
- **Early stopping:** Enabled with patience = 3
- **Loss tracking:** Training and validation loss curves are monitored to ensure convergence and avoid overfitting.
- **Checkpointing:** The best model checkpoint is automatically saved based on evaluation performance.

4.4 Training Processing

Experiment 1 — Qwen3VL (fine-tune 8.9K samples)

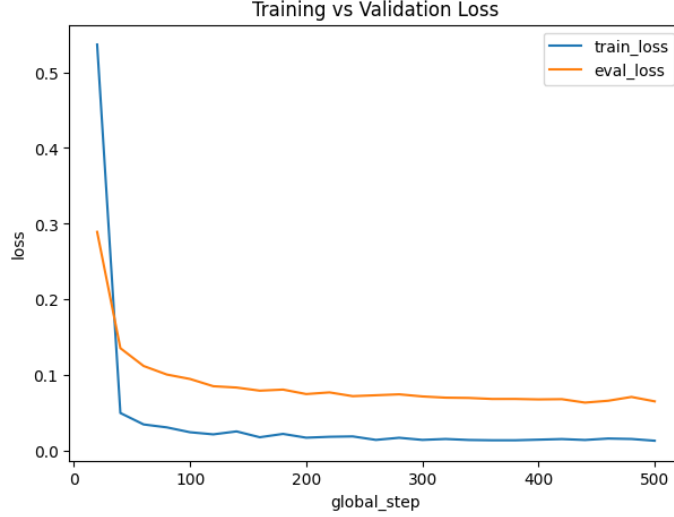


Fig. 15 Training vs Validation Loss (Experiment 1).

- The training loss decreased sharply during early iterations and stabilized near 0.05, while evaluation loss plateaued around 0.1, showing good convergence without overfitting.
- Training duration: 1 hour 42 minutes (500 steps).
- The model successfully learned to generalize across most handwritten symbol structures.

Experiment 2 — Qwen3VL (fine-tune 10.9K samples)

- With the inclusion of 2K augmented samples, the model achieved smoother loss curves and reduced evaluation loss (~ 0.06).
- Training duration: 1 hour 57 minutes (340/1315 steps).
- This suggests improved recognition stability and stronger generalization ability compared to the 8.9K-sample model.

Qwen3VL (Fine-tuned model):

The fine-tuned version demonstrates noticeable improvement. It correctly interprets most spatial and structural relationships, achieving closer alignment between the *Ground Truth Label Graph* and the *Predicted Label Graph*. This reflects a better comprehension of handwriting variations and mathematical layout patterns, confirming that fine-tuning significantly improves recognition accuracy.



Fig. 16 Training vs Validation Loss (Experiment 2).

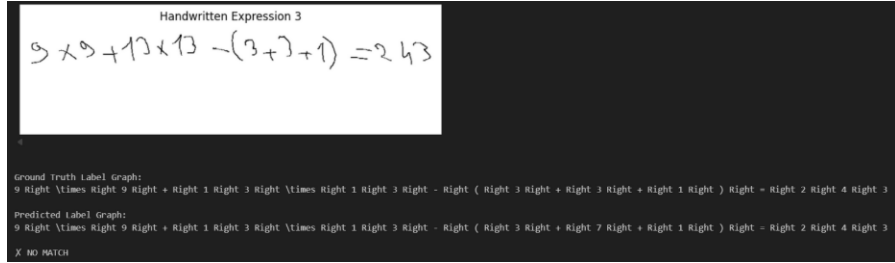


Fig. 17 Output of the fine-tuned Qwen3VL model showing correct prediction.

4.5 Conclusion

In general, Qwen3-VL outperformed the Vision Transformer Hybrid, showing superior learning efficiency and structural understanding of handwritten mathematical expressions.

By combining LoRA adapters, quantization, and multimodal attention layers, the fine-tuned Qwen3-VL achieved an optimal trade-off between computational efficiency and recognition accuracy — making it a promising model for the CROHME handwritten math recognition task.

5 Evaluating Model Performance

5.1 Explanation of BLEU Score

BLEU (which stands for *Bilingual Evaluation Understudy*) is an algorithm and metric originally designed to evaluate the quality of text automatically translated from one language to another [11].

In the context of this report, the BLEU score is used to measure the similarity between the $\text{\LaTeX}$ string generated by the model (the "candidate" text) and the correct reference $\text{\LaTeX}$ string (the "ground-truth" text) from the CROHME dataset. A higher score indicates a closer match between the model's output and the correct answer.

5.1.1 Mathematical Formulation

The BLEU score is computed as a geometric mean of modified n -gram precisions, multiplied by a Brevity Penalty to penalize translations that are too short [11].

The formula for the cumulative BLEU score is:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (1)$$

Where:

- N is the maximum n -gram order. For the final scores in this report, $N = 4$.
- p_n is the *modified n -gram precision*. It measures the proportion of n -grams (sequences of n tokens) in the candidate string that are also found in the reference string. The "modified" part involves "clipping" the count of each n -gram to the maximum number of times it appears in the reference, which prevents models from being rewarded for over-generating correct words.
- w_n is the weight for each n -gram precision. For the standard cumulative BLEU- N score, these are uniform, $w_n = 1/N$. For the BLEU-4 score, $w_1 = w_2 = w_3 = w_4 = 1/4$.
- BP is the *Brevity Penalty*. It is applied to penalize candidate strings that are shorter than the reference string. It is calculated as:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

where c is the length of the candidate string and r is the length of the reference string.

The detailed reports show the cumulative scores for each n -gram level (1-gram BLEU, 2-gram BLEU, etc.). The final score listed in the summary table (e.g., 77.67) is the cumulative 4-gram BLEU score, which incorporates the precision of all n -grams from 1 to 4.

In summary: The BLEU score ranges from 0 to 1 (or 0% to 100%). **The higher the score, the closer the model-generated $\text{\LaTeX}$ string is to the correct ground-truth $\text{\LaTeX}$ string.**

5.2 Analysis: Qwen3VL (Before Fine-Tuning)

This section details the performance of the baseline **Qwen3VL** model, prior to any fine-tuning. This model performed at a baseline level, with very low scores across the board, failing to correctly match any expression perfectly.

Summary Metrics

- **Model:** Qwen3VL
- **Total Expressions:** 1198
- **Exact Matches:** 0 (0.00%)
- **Overall BLEU Score (4-gram):** 1.32%

Detailed BLEU Scores

The N-gram BLEU scores show a sharp drop-off in performance as the complexity of the matching phrases (n-grams) increases.

- **1-gram BLEU:** 12.02%
- **2-gram BLEU:** 2.29%
- **3-gram BLEU:** 1.52%
- **4-gram BLEU:** 1.32%

These scores are visualized in the "N-gram BLEU Scores" bar chart (Figure 18), which shows the 1-gram score at approximately 0.12 (12.02%) and the other n-gram scores significantly lower, near 0.02 (2%).

Component-wise F1 Scores

The "Component-wise F1 Scores" chart (Figure 18) indicates how well the model recognized specific structural components of the $\text{\LaTeX}$. It performed best on 'Brackets' (~30.5%) but failed to identify 'subscripts' and 'superscripts' entirely.

Score Distribution

The "4-gram BLEU Score Distribution" histogram (Figure 18) shows that the vast majority of the test samples (over 600 out of 1198) received a BLEU score very close to 0.

5.3 Analysis: Qwen3VL (Fine-Tuned on CROHME)

This section provides a detailed analysis of the **Qwen3VL (fine-tuned)** model trained exclusively on the **CROHME (8.9k samples)** dataset. This model demonstrated the strongest performance of all evaluated models.

Summary Metrics

This model achieved the highest scores in both exact match accuracy and overall BLEU score.

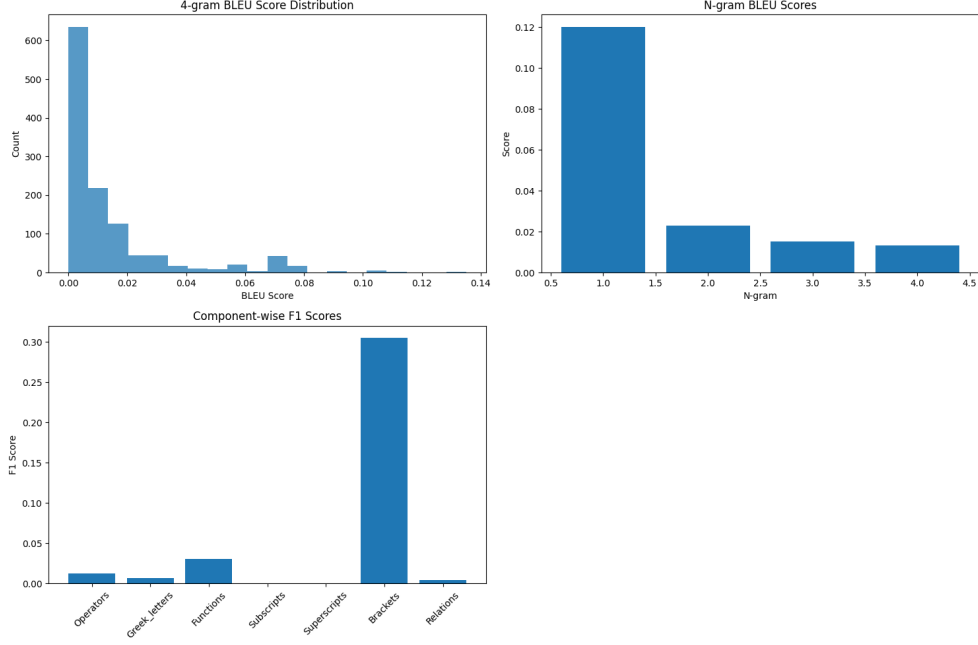


Fig. 18 Visualization of the baseline Qwen3VL model performance.

- **Model:** Qwen3VL (fine-tuned)
- **Training Dataset:** CROHME (8.9k samples)
- **Total Expressions:** 1198
- **Exact Matches:** 524 (43.74%)
- **Overall BLEU Score (4-gram):** 77.67%

Detailed BLEU Scores

The N-gram BLEU scores remain high even as the n -gram size increases, indicating the model is not just matching individual tokens but also correct phrases and structures.

- **1-gram BLEU:** 89.27%
- **2-gram BLEU:** 85.36%
- **3-gram BLEU:** 81.43%
- **4-gram BLEU:** 77.67%

Component-wise F1 Scores

The F1 scores (Figure 19) show the model’s effectiveness at identifying specific $\text{\LaTeX}$ components. The model performed best on ‘Brackets’ ($\sim 32.8\%$) and ‘Functions’ ($\sim 30.9\%$). Notably, both the HTML report and the plot show 0.00% F1 score for ‘subscripts’ and ‘superscripts’.

Score Distribution

The "4-gram BLEU Score Distribution" histogram (Figure 19) confirms the model's strong performance. There is a very large concentration of scores at the high end, with a significant spike of over 500 expressions receiving a BLEU score near 1.0. This aligns with the 524 (43.74%) exact matches reported.

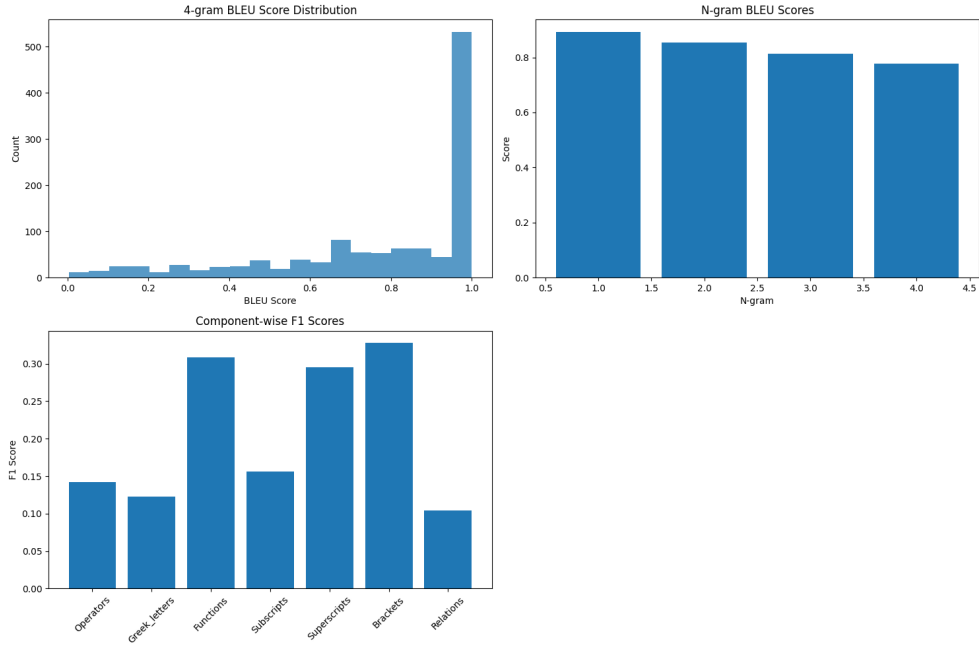


Fig. 19 Visualization of the Qwen3VL (fine-tuned) model performance.

5.4 Analysis: Qwen3VL (Fine-Tuned on CROHME + 2k datagen)

This section analyzes the **Qwen3VL (fine-tuned)** model trained on the augmented dataset, **CROHME (8.9k samples) + 2k datagen (10.9k samples)**. Adding 2,000 generated samples resulted in a slight decrease in performance compared to the model trained only on the 8.9k CROHME set.

Summary Metrics

- **Model:** Qwen3VL (fine-tuned)
- **Training Dataset:** CROHME + 2k datagen (10.9k samples)
- **Total Expressions:** 1198
- **Exact Matches:** 507 (42.32%)
- **Overall BLEU Score (4-gram):** 75.46%

Detailed BLEU Scores

The N-gram BLEU scores remain high, though slightly lower than the 8.9k-only model.

- **1-gram BLEU:** 87.56%
- **2-gram BLEU:** 82.79%
- **3-gram BLEU:** 78.96%
- **4-gram BLEU:** 75.46%

Component-wise F1 Scores

The F1 scores are visualized in Figure 20. Interestingly, there is a discrepancy: the HTML report indicates 0.00% F1 for ‘subscripts’ and ‘superscripts’, but the plot clearly shows high scores for them (‘Superscripts’ $\sim 31.0\%$, ‘Subscripts’ $\sim 17.5\%$). The scores for other components like ‘Brackets’ ($\sim 32.0\%$) and ‘Functions’ ($\sim 31.4\%$) remain high.

Score Distribution

The “4-gram BLEU Score Distribution” histogram (Figure 20) is very similar to the 8.9k model, with a large peak of over 500 expressions receiving a BLEU score near 1.0. This aligns with the 507 (42.32%) exact matches reported.

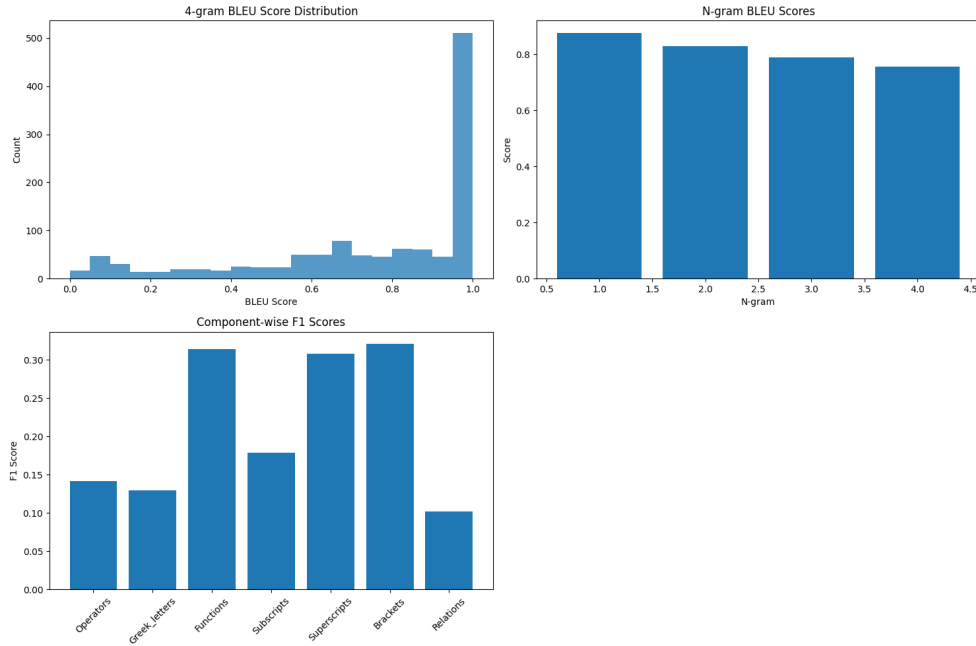


Fig. 20 Visualization of the Qwen3VL (fine-tuned) model performance with 2k datagen.

5.5 Model Performance Comparison on CROHME Dataset

Here is a summary of the model evaluation results presented in the data. This table compares different models based on their training dataset, exact match accuracy, and BLEU scores.

Table 3 Model Performance Comparison on CROHME Dataset

Model	Train dataset	Match accuracy (%)	BLEU
GC Block (iViT + Transformer)	CROHME (8.9k)	4.92% (59/1198)	43.26
Qwen3VL [7]	CROHME (8.9k)	0% (0/1198)	1.32
Qwen2.5VL (fine-tuned)	CROHME (8.9k)	28.38% (340/1198)	63.00
Qwen3VL (fine-tuned)	CROHME (8.9k)	43.74% (524/1198)	77.67
Qwen3VL (fine-tuned)	CROHME + 2k (10.9k)	42.32% (507/1198)	75.46
IM2TEX [12]	CROHME (8.9k)	38.74% (433/1198)	50.28

5.6 Conclusion

Based on the evaluation data, the **Qwen3VL (fine-tuned)** model trained exclusively on the **CROHME (8.9k)** dataset demonstrates the strongest performance. It achieves the highest **Match accuracy at 43.74% (524/1198)** and the top **BLEU score of 77.67**.

This performance highlights the critical impact of fine-tuning, as the baseline Qwen3VL model scored 0% accuracy with a 1.32 BLEU score. The fine-tuned Qwen3VL also significantly surpasses all other models in the comparison, including the fine-tuned Qwen2.5VL (28.38% accuracy) and IM2TEX (38.74% accuracy).

Interestingly, augmenting the training data with 2,000 generated samples (+ 2k datagen) resulted in a slight decrease in performance for the Qwen3VL model, with accuracy dropping to 42.32% and the BLEU score to 75.46.

References

- [1] Zanibbi, R., & Blostein, D. (2012). Recognition and retrieval of mathematical expressions. *International Journal on Document Analysis and Recognition (IJDAR)*, 15(4), 331-357.
- [2] Mahdavi, M., Zanibbi, R., Mouchère, H., & Garain, U. (2019). ICDAR 2019 competition on recognition of handwritten mathematical expressions and type-set formula detection. *15th International Conference on Document Analysis and Recognition (ICDAR)*.
- [3] W3C, *InkML: Ink Markup Language (W3C Recommendation)*, 2003. Available: <https://www.w3.org/2003/InkML>
- [4] Rednote HiLab, *dots.ocr: Multilingual Document Layout Parsing in a Single Vision-Language Model*, GitHub repository, 2024. Available: <https://github.com/rednote-hilab/dots.ocr>

- [5] Dosovitskiy, A., et al. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.
- [6] Kolesnikov, A., et al. (2020). Big transfer (BiT): General visual representation learning. *Computer Vision–ECCV 2020*, 491-507.
- [7] Alibaba Group, *Qwen3-VL: Vision-Language Model by QwenLM*, GitHub repository, 2024. Available: <https://github.com/QwenLM/Qwen3-VL>
- [8] UnslothAI, *Unsloth: Fine-tuning LLMs efficiently*, GitHub repository, 2024. Available: <https://github.com/unslothai/unsloth>
- [9] Hu, E. J., et al. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- [10] Dettmers, T., et al. (2023). QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36.
- [11] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a method for automatic evaluation of machine translation. *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 311-318.
- [12] Deng, Y., et al. (2016). Image-to-markup generation with coarse-to-fine attention. *International Conference on Machine Learning (ICML)*.